CURS:	DAM2	**DATA:**	Octubre-2026	**Qualificació**	
MÒDUL:	M0490 - Programació Serveis i Processos				
TEMA:	Criptografia - Integritat				
RA	RA5				
Professors	Cristina Reina - Carles Bonet				
Alumne	Albert Floriach Calzón				

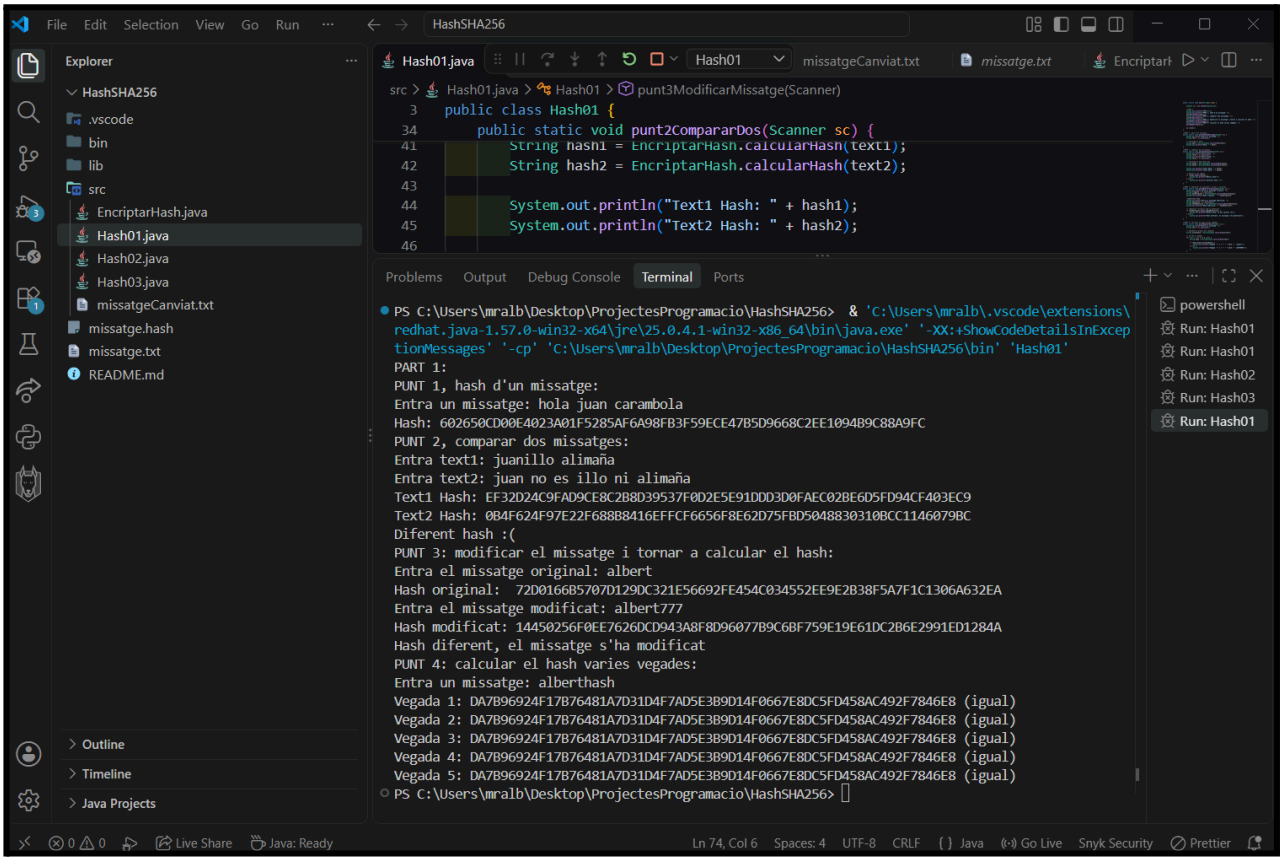
Part 1: Càlcul de resums

Desenvolupa un programa que permeti introduir missatges i calcular-ne el hash mitjançant l'algorisme SHA-256.

El programa ha de permetre:

1. Introduir un missatge per teclat i mostrar-ne el hash.
2. Introduir dos missatges i comprovar si els seus hashes coincideixen.
3. Calcular el hash d'un missatge, modificar-ne el contingut i tornar-lo a calcular per observar-ne les diferències.
4. Calcular diverses vegades el hash d'un mateix missatge i comprovar si el resultat es manté.

Els hashes s'han de mostrar en format hexadecimal.



Es pot veure que el mateix missatge sempre dona el mateix hash i que canviant una lletra el hash

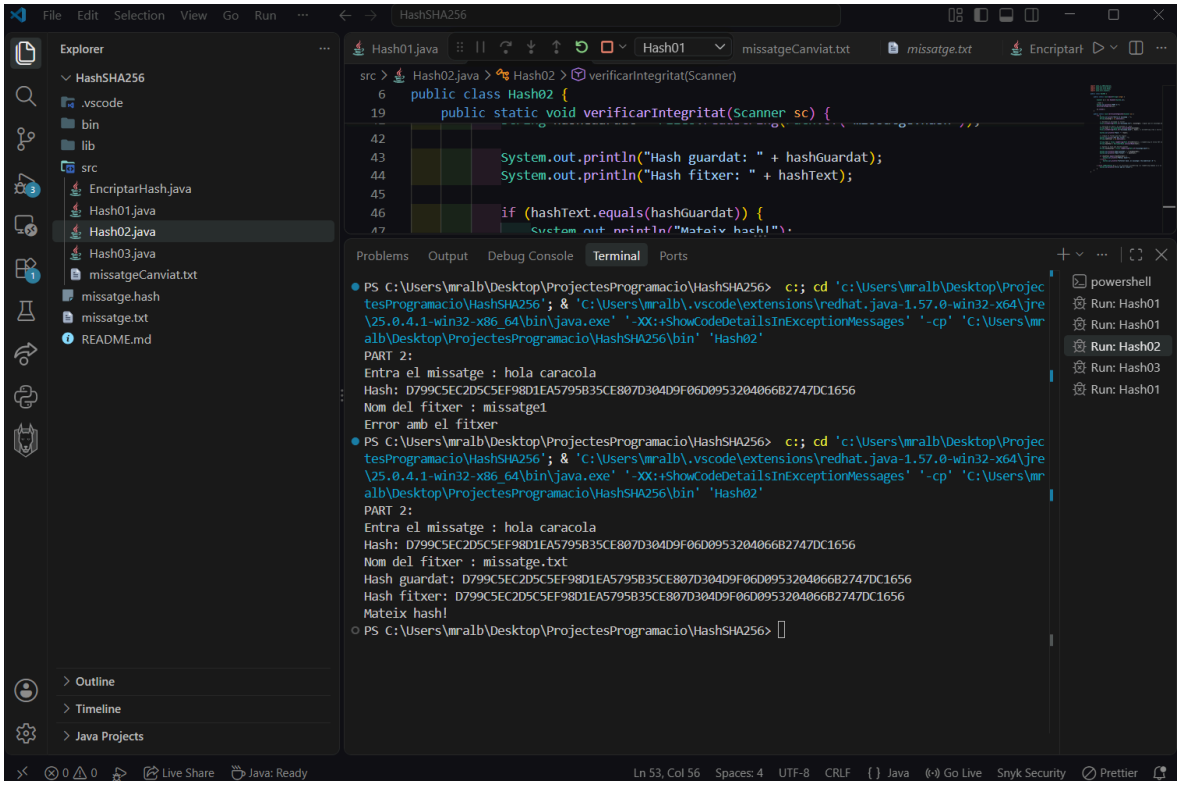
canvia completament.

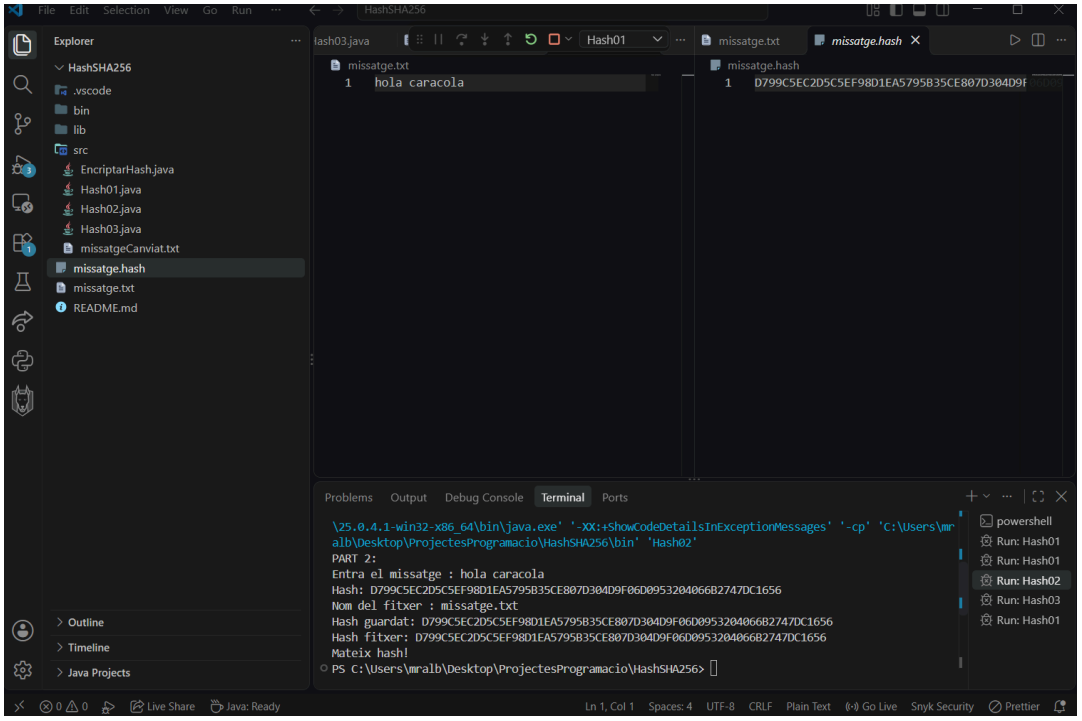
Part 2: Verificació de la integritat dels missatges

Implementa una funcionalitat que simuli l'enviament d'un missatge entre dos usuaris.

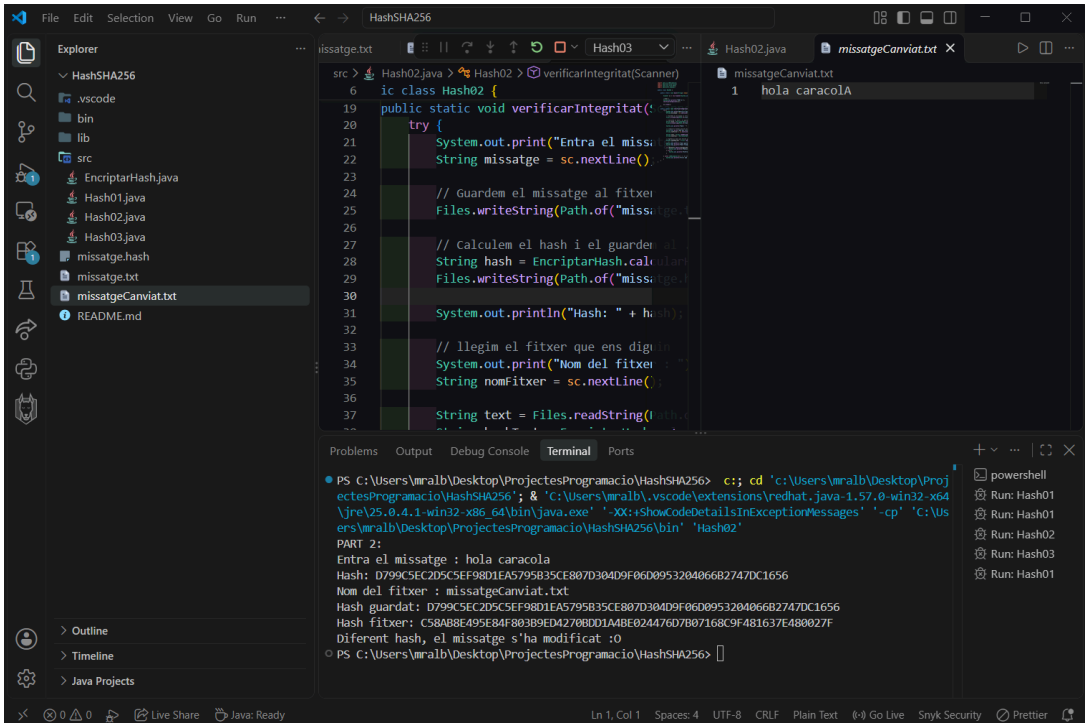
El programa haurà de:

1. Demanar a l'emissor el contingut d'un missatge i el desarà en un fitxer "missatge.txt".
2. Calcular i guardar temporalment el hash original en un fitxer "missatge.hash".
3. Simular la recepció del missatge.
4. Permetre introduir el missatge rebut, que podrà ser igual o diferent de l'original: Introduir el nom del fitxer a llegir des de teclat. Pot ser el fitxer original "missatge.txt" o el "missatgeCanviat.txt" (que conté una versió modificada de l'original.
5. Calcular el hash del missatge rebut (fitxer indicat) i comparar-lo amb el hash original (missatge.hash).
6. Indicar si els dos hashes coincideixen o si s'ha detectat una modificació.





Podem veure que writeString i readString han fet la seva funció i han guardat el missatge/llegit en missatge.txt



Com "missatgeCanviat.txt" tenia "hola caracolA" al posar "hola caracola" diu que el hash del fitxer rebut no coincideix amb el guardat a missatge.hash. Per això el programa mostra "Diferent hash, el missatge s'ha modificat", és a dir, detecta que el missatge que ha arribat no és igual que l'original.

Aquesta part ha de permetre comprovar experimentalment com una modificació del missatge afecta el seu hash.

Part 3: Registre d'usuaris amb salt

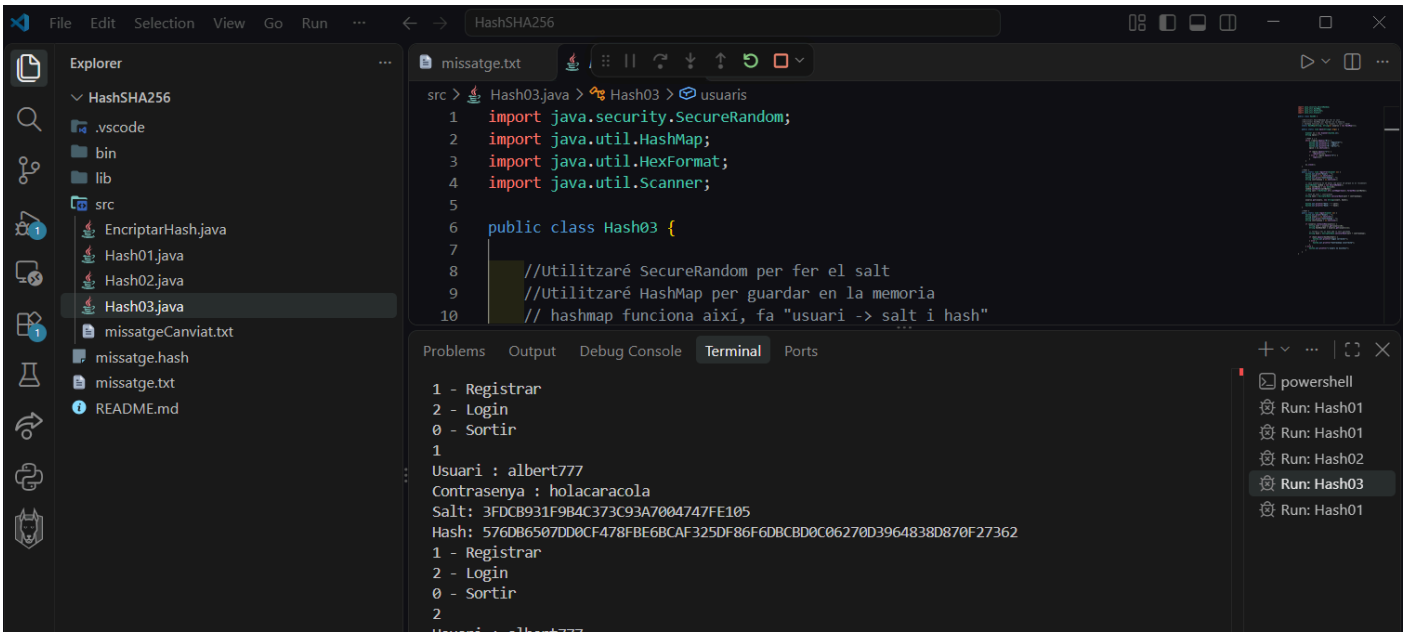
Desenvolupa un sistema de registre que permeti crear usuaris i emmagatzemar les seves credencials de manera que les contrasenyes originals no es guardin en text pla.

Per a cada usuari, el programa haurà de:

1. Demanar un nom d'usuari i una contrasenya.
2. Generar una salt aleatòria i diferent per a cada registre.
3. Calcular el hash de la contrasenya utilitzant la salt generada.
4. Emmagatzemar el nom d'usuari, la salt i el hash resultant.

Les dades es poden guardar inicialment en una estructura de dades en memòria.

No s'ha de guardar ni mostrar la contrasenya original una vegada finalitzat el registre.



Podem veure que ha registat al usuari correctament i li ha fet el Hash + Salt.

Part 4: Inici de sessió i verificació de contrasenyes

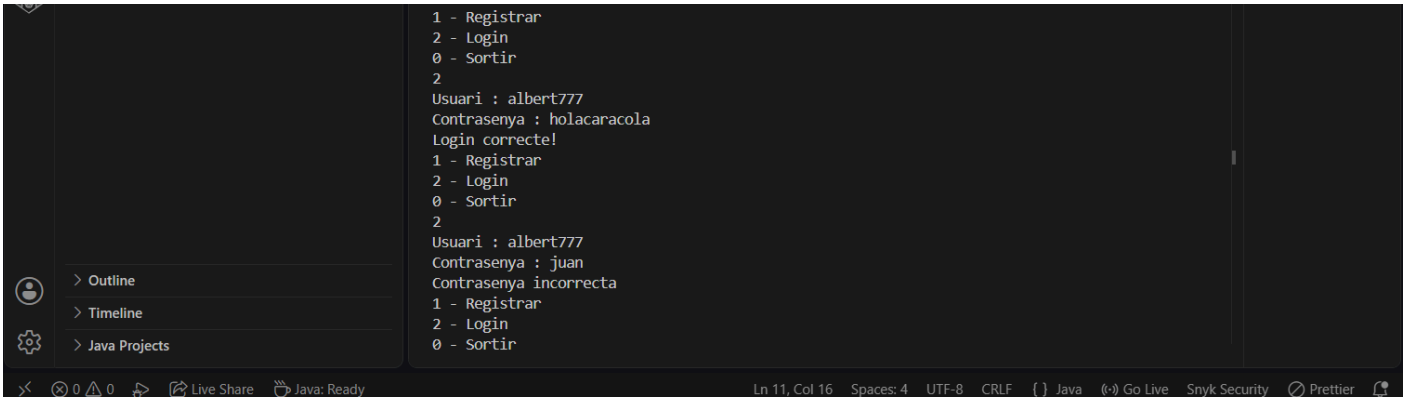
Implementa un sistema d'autenticació que permeti als usuaris registrats iniciar sessió.

El programa haurà de:

1. Demanar el nom d'usuari i la contrasenya.
2. Comprovar si l'usuari està registrat.
3. Recuperar la salt i el hash associats a l'usuari.
4. Calcular el hash de la contrasenya introduïda utilitzant la salt emmagatzemada.

5. Comparar el hash calculat amb el hash guardat.
6. Permetre l'accés únicament quan els hashes coincideixin.

Si les credencials no són correctes, el programa haurà de mostrar un missatge d'error adequat.



Es pot apreciar que un login correcte ho detecta i un login incorrecte efectivament també ho detecta.

Part 5: Proves de la pràctica

Realitza les proves següents i documenta'n els resultats:

- Registra dos usuaris diferents amb la mateixa contrasenya. Comprova si obtenen hashes diferents.
- Intenta iniciar sessió amb una contrasenya correcta.
- Intenta iniciar sessió amb una contrasenya incorrecta.
- Comprova què passa quan s'utilitza una salt diferent de la que s'ha guardat.
- Explica per què les salts han de ser diferents per als diferents registres i per què cal conservar la salt de cada usuari.

```
Usuari : usuari1
Contrasenya : hola
Salt: 5D0C1544C9DEEDCEAE101B6C25EC8C64
Hash: 9CDBC5950DD6B833E719FD49238A4866C69187104FFE2E0891241320B64F7773
1 - Registrar
2 - Login
0 - Sortir
1
Usuari : usuari2
Contrasenya : hola
Salt: A2194DBE060C9CBB9CCDF40C98E8E706
Hash: D53B026990C7286F81004EE925BE9D0AF59FE14708CF5445B2655374E2341607
1 - Registrar
2 - Login
0 - Sortir
```

Gràcies a SecureRandom aconseguim que 2 usuaris puguin tenir la mateixa contrasenya, però el hash diferent gràcies al salt.

Iniciar sessió amb una contrasenya correcta i incorrecta

Es pot veure a la captura de la Part 4: amb la contrasenya correcta "holacaracola" surt Login correcte i amb una contrasenya incorrecta "juan" surt "Contrasenya incorrecta".

Salt diferent de la guardada

Si s'utilitza una salt diferent de la que s'ha guardat, el hash que surt és diferent del guardat, i per tant el login falla encara que la contrasenya sigui la correcta.

Per què les salts han de ser diferents i per què cal conservar-les

Les salts han de ser diferents perquè el mateix text sempre dona el mateix hash. Sense salt, dos usuaris amb la mateixa contrasenya tindran el mateix hash i es veuria que la comparteixen. També hi ha llistes amb els hashes de les contrasenyes més típiques i es podria descobrir la contrasenya. Amb una salt diferent per a cada usuari això no passa, com es veu amb usuari1 i usuari2.

Cal conservar la salt perquè al fer login s'ha de tornar a calcular el hash amb la mateixa salt + la contrasenya introduïda. Si no la guardéssim, no es podria tornar a obtenir el mateix hash i ningú podria iniciar sessió.